

Taller de TDA

Estudiante

Cristian Marquez Arcila

Docente

Flavio Alexander Navarro Carmona

Fundación Universitaria Empresarial de la Cámara de Comercio de
Bogotá - Uniempresarial

Ingeniería de Software

Estructura de datos

Colombia, Bogotá D.C.

Contenido

Introducción	3
Desarrollo.....	4
Parte 1 conceptual.....	4
Parte 2: Análisis y Aplicación	7
Parte 3: Python	9

Introducción

En el desarrollo de software, la complejidad de los sistemas crece a medida que aumentan sus funcionalidades. Para manejar esta complejidad, la programación moderna se apoya en conceptos fundamentales como los Tipos de Datos Abstractos (TDA) y la Modularidad. Un TDA permite definir estructuras de datos a través de su comportamiento y operaciones, ocultando los detalles internos de implementación, lo que facilita la reutilización y el mantenimiento del código. Por otro lado, la modularidad consiste en dividir un programa en componentes independientes y cohesivos que se comunican mediante interfaces claras, reduciendo el acoplamiento y mejorando la organización del software. Este taller explora ambos conceptos desde una perspectiva teórica y práctica, incluyendo su aplicación en el lenguaje Python, con el objetivo de comprender cómo la abstracción y la separación de responsabilidades contribuyen a construir sistemas más robustos, escalables y fáciles de mantener.

Desarrollo

Parte 1 conceptual

1. ¿Qué es un Tipo de Dato Abstracto (TDA)?

Es un modelo matemático que define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, ocultando los detalles de implementación. El usuario solo conoce qué hace, no cómo lo hace.

2. ¿Cuál es la diferencia entre TDA e implementación?

El TDA es la especificación (qué operaciones se pueden hacer y qué datos se manejan), mientras que la implementación es el código concreto que realiza esas operaciones. Un mismo TDA puede tener múltiples implementaciones.

3. ¿Por qué se dice que un TDA define el "qué" y no el "cómo"?

Porque el TDA solo especifica la interfaz (operaciones disponibles y su comportamiento), sin detallar la estructura interna ni los algoritmos usados para implementarlas.

La Interfaz (El "Qué"): La parte visible del TDA. Define las operaciones permitidas (añadir, eliminar, buscar, etc.) y cómo interactuar con ellas, sin revelar el código fuente ni la estructura interna.

La Implementación (El "Cómo"): El código oculto que hace que el TDA funcione. Es la lógica real (generalmente usando arreglos o listas) que ejecuta las acciones definidas en la interfaz.

4. Mencione 3 ejemplos de TDA.

- Pila (Stack): operaciones push, pop, peek, isEmpty.
- Cola (Queue): operaciones enqueue, dequeue, front, isEmpty.
- Lista (List): operaciones insert, delete, search, size.

5. ¿Qué es una operación dentro de un TDA?

Es una función que puede realizarse sobre los datos del TDA, definida por su nombre, parámetros, precondiciones y postcondiciones. Ejemplo: en una pila, push(elemento) agrega un elemento en la cima.

6. ¿Qué relación existe entre estructuras de datos y TDA?

Un TDA es la abstracción (qué hace), mientras que la estructura de datos es la implementación concreta (cómo lo hace). Por ejemplo, el TDA "pila" puede implementarse con un arreglo o con una lista enlazada.

7. ¿Por qué es importante el uso de TDA en programación?

Porque permite separar la interfaz de la implementación, facilitando el mantenimiento, la reutilización del código, y reduciendo la complejidad al permitir trabajar con abstracciones.

8. ¿Qué es encapsulamiento en el contexto de TDA?

Es ocultar los detalles internos de implementación y exponer solo una interfaz pública. Los datos internos no son accesibles directamente desde fuera del TDA.

9. ¿Qué ventajas ofrece trabajar con abstracción?

- Reduce la complejidad.
- Facilita el mantenimiento.
- Permite reutilizar código.
- Mejora la legibilidad.

10. ¿Qué problemas se presentan si no se usa abstracción?

- Código difícil de mantener y modificar.
- Alta dependencia entre componentes.
- Dificultad para reutilizar código.
- Mayor probabilidad de errores al cambiar implementaciones.
- Código más extenso y difícil de entender.

11. ¿Qué es modularidad?

Es la técnica de dividir un programa en partes independientes llamadas módulos, cada uno con una responsabilidad bien definida, que se comunican entre sí a través de interfaces.

12. ¿Por qué es importante dividir un programa en módulos?

- Facilita el mantenimiento (cambiar un módulo sin afectar otros).
- Permite el trabajo en equipo (cada persona trabaja en un módulo).
- Favorece la reutilización.
- Reduce la complejidad general del sistema.

13. ¿Qué es cohesión en un módulo?

Es el grado en que los elementos dentro de un módulo están relacionados entre sí. Un módulo con alta cohesión agrupa funciones y datos que tienen un propósito único y bien definido.

14. ¿Qué es acoplamiento?

Es el grado de dependencia entre módulos. Un bajo acoplamiento significa que los módulos son independientes y se comunican solo a través de interfaces simples, lo cual es deseable.

15. ¿Cómo se relaciona la modularidad con el mantenimiento del software?

La modularidad facilita el mantenimiento porque permite modificar, reparar o actualizar un módulo sin afectar al resto del sistema. También facilita encontrar errores y agregar nuevas funcionalidades.

Parte 2: Análisis y Aplicación

16. Diseñe un TDA para una pila (stack) indicando sus operaciones.

- push(elemento) → insertar elemento.
- pop() → eliminar elemento superior.
- peek() → ver elemento superior.
- isEmpty() → verificar si está vacía.

17. Diseñe un TDA para una cola (queue).

- enqueue(elemento) → agregar elemento.
- dequeue() → eliminar primer elemento.
- front() → ver primer elemento.
- isEmpty() → verificar si está vacía.

18. Diseñe un TDA para un carrito de compras.

- agregarProducto(producto)
- eliminarProducto(producto)
- calcularTotal()
- mostrarProductos()
- vaciarCarrito()

19. Explique cómo se aplicaría modularidad en un sistema de estudiantes.

- Registro de estudiantes.
- Gestión de notas.
- Matrículas.
- Reportes.
- Usuarios y autenticación.

Cada módulo tendría funciones independientes.

20. ¿Qué pasaría si todo el código de un sistema se hace en un solo archivo?

- Sería extremadamente difícil de mantener y entender.
- Cualquier cambio podría afectar todo el sistema.
- Imposible trabajar en equipo de forma eficiente.
- Dificultad para reutilizar código.
- Mayor probabilidad de errores.

El código sería difícil de entender, mantener y corregir. Además, aumentaría la probabilidad de errores.

21. Dé un ejemplo donde el TDA sea independiente de la implementación.

Una pila puede funcionar igual para el usuario, aunque internamente esté implementada con arreglos o listas enlazadas.

22. ¿Por qué una pila puede implementarse con lista o arreglo?

Porque ambos permiten almacenar elementos y acceder al último insertado, cumpliendo el comportamiento LIFO.

23. Diseñe un módulo para manejar usuarios en un sistema.

- crearUsuario()
- eliminarUsuario()
- actualizarUsuario()
- buscarUsuario()
- listarUsuarios()

24. ¿Qué ventajas tiene separar funciones en diferentes módulos?

- Mejor organización.
- Mayor reutilización.
- Fácil mantenimiento.
- Trabajo en equipo más sencillo.

25. Explique con un ejemplo la diferencia entre alta cohesión y bajo acoplamiento

Alta cohesión: un módulo (Facturacion) solo contiene funciones relacionadas con facturas (crearFactura, anularFactura, calcularImpuestos). Todo está relacionado con un solo propósito.

Bajo acoplamiento: el módulo (Facturacion) se comunica con "Inventario" solo mediante una función "descontarStock(producto, cantidad)". Si cambia la implementación interna de "Inventario", "Facturacion" no se ve afectada siempre que la interfaz se mantenga.

Parte 3: Python

26. Cree una clase en Python que represente una pila (push y pop).

```
Tda.py > ...
1  class Pila:
2      def __init__(self):
3          self.elementos = []
4
5      def push(self, elemento):
6          self.elementos.append(elemento)
7
8      def pop(self):
9          if len(self.elementos) > 0:
10             return self.elementos.pop()
11             return "La pila está vacía"
12
13  # Ejemplo
14  p = Pila()
15  p.push(10)
16  p.push(20)
17
18  print(p.pop())
```

27. Cree una función en Python que represente un módulo para agregar elementos a una lista.

```
Tda.py > ...
1 def agregar_elemento(lista, elemento):
2     lista.append(elemento)
3
4 # Ejemplo
5 datos = [1, 2, 3]
6 agregar_elemento(datos, 4)
7
8 print(datos)
```

28. Cree un ejemplo de modularidad separando funciones en Python.

Modulo main.py

```
ESTRUCTURA_DATOS
> __pycache__
main.py
modulo_operaciones.py
Taller_TDA_Modularidad.md
Tda.py

main.py
1 #Modulo main
2 from modulo_operaciones import suma, resta
3
4 print(f'El resultado de la suma es: {suma(5, 3)}')
5 print(f'El resultado de la resta es: {resta(5, 3)}')
```

Este módulo es el encargado de la parte visual e interacción con el usuario. Desde aquí se llaman las funciones del módulo modulo_operaciones.py para mostrar resultados o ejecutar las operaciones necesarias.

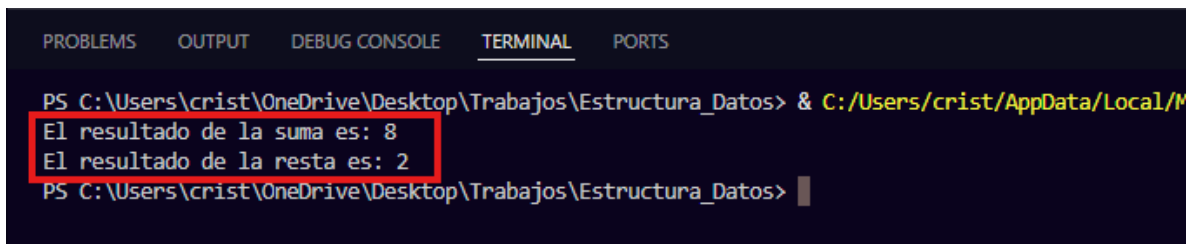
Modulo_operaciones.py

```
ESTRUCTURA_DATOS
> __pycache__
main.py
modulo_operaciones.py
Taller_TDA_Modularidad.md
Tda.py

modulo_operaciones.py > resta
1 # modulo_operaciones.py
2 def suma(a, b):
3     return a + b
4
5 def resta(a, b):
6     return a - b
```

Este módulo contiene las funciones encargadas de realizar las operaciones matemáticas, como suma y resta. Su función es manejar la lógica del programa, separando los cálculos de la parte visual.

Consola



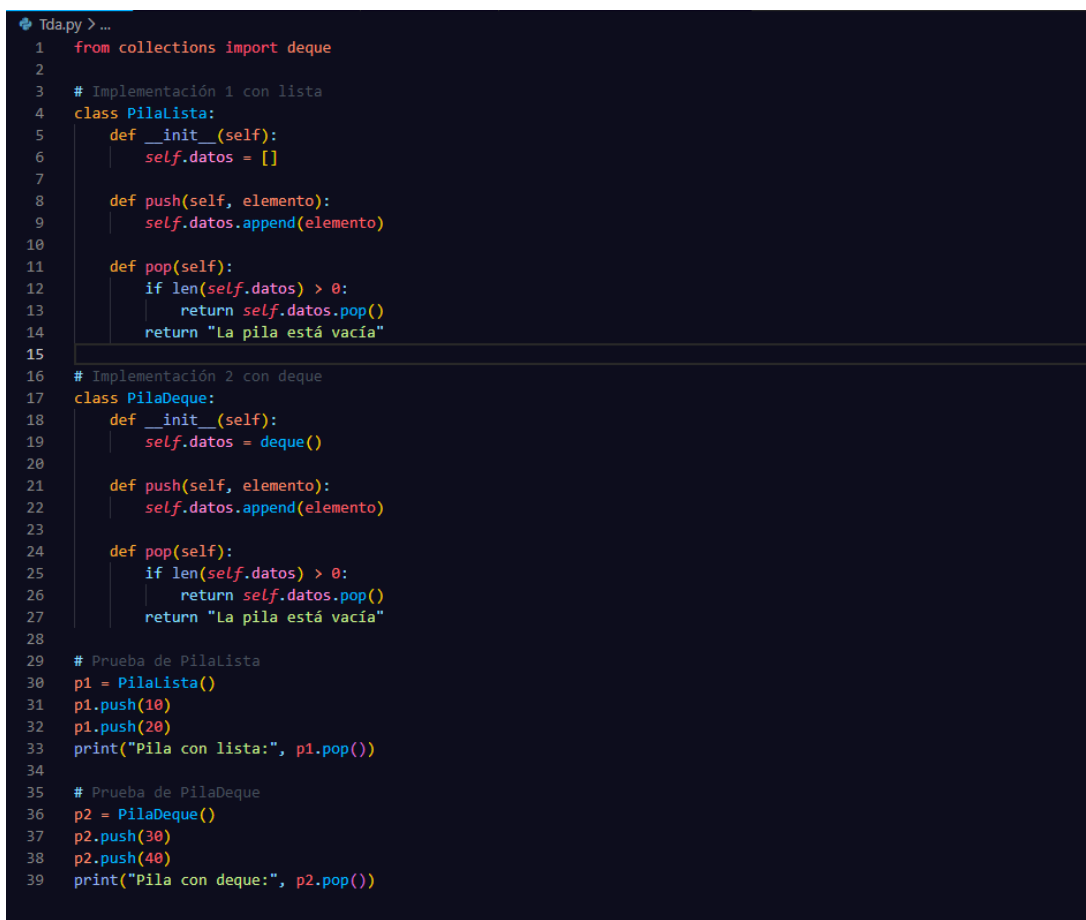
```
PS C:\Users\cris\OneDrive\Desktop\Trabajos\Estructura_Datos> & C:/Users/crist/AppData/Local/M
El resultado de la suma es: 8
El resultado de la resta es: 2
PS C:\Users\cris\OneDrive\Desktop\Trabajos\Estructura_Datos> |
```

Resultados de la ejecución del Código.

29. Explique cómo Python permite implementar un TDA.

Python permite implementar TDA mediante clases, métodos y encapsulamiento, ocultando detalles internos y mostrando solo las operaciones necesarias.

30. Escriba un ejemplo donde un mismo TDA tenga dos implementaciones diferentes.



```
Tda.py > ...
1  from collections import deque
2
3  # Implementación 1 con lista
4  class PilaLista:
5      def __init__(self):
6          self.datos = []
7
8      def push(self, elemento):
9          self.datos.append(elemento)
10
11     def pop(self):
12         if len(self.datos) > 0:
13             return self.datos.pop()
14         return "La pila está vacía"
15
16 # Implementación 2 con deque
17 class PilaDeque:
18     def __init__(self):
19         self.datos = deque()
20
21     def push(self, elemento):
22         self.datos.append(elemento)
23
24     def pop(self):
25         if len(self.datos) > 0:
26             return self.datos.pop()
27         return "La pila está vacía"
28
29 # Prueba de PilaLista
30 p1 = PilaLista()
31 p1.push(10)
32 p1.push(20)
33 print("Pila con lista:", p1.pop())
34
35 # Prueba de PilaDeque
36 p2 = PilaDeque()
37 p2.push(30)
38 p2.push(40)
39 print("Pila con deque:", p2.pop())
```